The Myra Programming Language -- BNF

Syntax Notation

EBNF notation is used. Brackets `[` and `]` denote optionality. Braces `{` and `}` denote repetition (zero or more). Parentheses `(` and `)` group alternatives. The vertical bar `|` separates alternatives. Terminal symbols are enclosed in quotes or written in lowercase. Non-terminals are written in PascalCase.

> **Design Principle:** Myra is a Pascal/Oberon-inspired systems programming language that compiles to C++ 23 via Metamorf. All languages built with Metamorf inherit seamless C++ passthrough: C++ keywords, operators, preprocessor directives, and block constructs pass through to the C++ backend unmodified. To keep the lexer's token assignment unambiguous, **no Myra keyword collides with any C++ keyword.** Clang (via Zig) handles all C++ validation.

1. Lexical Elements

```text
letter     = "A" | ... | "Z" | "a" | ... | "z" | "_" .
digit      = "0" | ... | "9" .
hexDigit   = digit | "A" | ... | "F" | "a" | ... | "f" .
character  = (* any source character except the delimiter *) .
newline    = (* line feed (U+000A) *) .

ident      = letter { letter | digit } .
integer    = digit { digit } | "0" ( "x" | "X" ) hexDigit { hexDigit } .
float_literal = digit { digit } "." { digit } [ exponent ] [ "f" | "F" ] .
exponent      = ( "e" | "E" ) [ "+" | "-" ] digit { digit } .
cstring    = '"' { character | escapeSeq } '"' .
wstring    = "w" '"' { character | escapeSeq } '"' .
escapeSeq  = "\" ( "n" | "t" | "r" | "0" | "\" | "'" | '"' | "x" hexDigit hexDigit ) .
```

Numeric Literal Type Rules

Literal	Suffix	Type	C++ Emit	Example
`42`	-	`int32`	`42`	integer
`1.5`	-	contextual	see below	float_literal
`1.5f`, `1.5F`	f/F	`float32`	`1.5f`	explicit float32

Float literal resolution (no suffix):

- If assigned to a `float32` variable or passed to a `float32` parameter: `float32`, emits `1.5f`
- If assigned to a `float64` variable or passed to a `float64` parameter: `float64`, emits `1.5`
- If context is ambiguous or unknown: `float64` (safe default, wider type)

Float literal resolution (f/F suffix):

- Always `float32` regardless of context, emits with `f` suffix

String Literal Convention

- `"..."` -- C-string literal (`const char*`). Escape sequences processed. Used for C++ interop.
- `w"..."` -- Wide-string literal (`std::wstring`). Escape sequences processed. Prefix is case-sensitive: only lowercase `w`.

2. Reserved Words

The language is **case-sensitive** for keywords and identifiers.

```text
address     align       and         array       begin       choices
const       create      destroy     div         do          downto
else        end         except      exported    external    false
finally     for         freeMem     getExceptionCode
getExceptionMessage     getMem      guard       if          import
in          is          len         match       method      mod         module
nil         not         object      of          or          overlay
packed      paramCount  paramStr    parent      pointer
raiseException          raiseExceptionCode      record      repeat
resizeMem   return      routine     self        set         setLength
shl         shr         size        test        then        to          true
testAssert              testAssertTrue          testAssertFalse
testAssertEqualInt      testAssertEqualUInt     testAssertEqualFloat
testAssertEqualStr      testAssertEqualBool     testAssertEqualPtr
testAssertNil           testAssertNotNil        testFail
type        until       utf8        var         varargs     while
write       writeln     xor
```

> **Note:** The identifiers `exe`, `dll`, and `lib` are contextual -- they have special meaning only in the `ModuleKind` position and may be used as ordinary identifiers elsewhere.

> **Design Note:** Every C++ keyword (`class`, `enum`, `struct`, `union`, `switch`, `template`, etc.) is deliberately avoided so the lexer can unambiguously assign every token to either the Myra or C++ family with zero context needed. Where C++ has a keyword, Myra uses an alternative: `object` instead of `class`, `choices` instead of `enum`, `overlay` instead of `union`, `match` instead of `switch`, `size` instead of `sizeof`.

3. Built-in Types

```text
int8        int16       int32       int64
uint8       uint16      uint32      uint64
float32     float64
boolean
char        wchar
string      wstring
pointer
```

Type Mapping to C++23

Myra Type	C++ Type
`string`	`std::string`
`wstring`	`std::wstring`
`int8`	`int8_t`
`int16`	`int16_t`
`int32`	`int32_t`
`int64`	`int64_t`
`uint8`	`uint8_t`

Myra Type	C++ Type
`uint16`	`uint16_t`
`uint32`	`uint32_t`
`uint64`	`uint64_t`
`float32`	`float`
`float64`	`double`
`boolean`	`bool`
`char`	`char`
`wchar`	`wchar_t`
`pointer`	`void*`

> Unknown type names pass through as-is to C++ output (e.g., `std::vector<int32>` written directly in Myra source reaches clang unmodified).

4. Operators and Delimiters

```text
+    -    *    /    =    <>   <    >    <=   >=
:=   +=   -=   *=   /=
:    ;    ,    .    ..   ...  ^    |    &
(    )    [    ]
```

Operator Semantics

- `:=` -- Assignment (emits C++ `=`)
- `=` -- Comparison (emits C++ `==`)
- `<>` -- Not equal (emits C++ `!=`)
- `^` -- Postfix: pointer dereference
- `|` and `&` -- Reserved tokens, not used by Myra grammar (available for future use)

5. Comments

```text
Comment    = "//" { character } newline
           | "/*" { character | Comment } "*/" .
```

- `//` -- Line comment (shared with C++).
- `/* ... */` -- Block comment (shared with C++, passes through naturally).

> **Note:** `(* *)` is NOT a comment delimiter -- it conflicts with C++ dereference `(*ptr)`.
> `{ }` is NOT a comment delimiter -- it is a C++ block delimiter.

6. Module Structure

```text
ModuleFile    = Module { TestBlock } .

Module        = "module" ModuleKind ident ";" [ Directives ] [ ImportClause ]
                { Declaration } [ "begin" StatementSeq ] "end" "." .

TestBlock     = "test" cstring [ VarBlock ] "begin" StatementSeq "end" ";" .

ModuleKind    = "exe" | "dll" | "lib" .

Directives    = { Directive } .
Directive     = "@" ident [ DirectiveValue ] .
DirectiveValue = cstring | integer | float_literal | ident .

ImportClause  = "import" ident { "," ident } ";" .
```

7. Conditional Compilation

```text
ConditionalDirective = DefineDir | UndefDir | IfdefDir | IfndefDir
                     | ElseIfDir | ElseDir | EndifDir .

DefineDir   = "@define" ident .
UndefDir    = "@undef" ident .
IfdefDir    = "@ifdef" ident .
IfndefDir   = "@ifndef" ident .
ElseIfDir   = "@elseif" ident .
ElseDir     = "@else" .
EndifDir    = "@endif" .
```

Known Directives

Module-level directives (appear after `module` header, before or among declarations):

- `@exeIcon "path"` -- Sets the application icon (Windows EXE modules only).
- `@copyDll "path"` -- Copies a DLL to the output directory during build.
- `@linkLibrary "path"` -- Tells Zig build which library to link against.
- `@libraryPath "path"` -- Adds a directory to the linker library search path.
- `@modulePath "path"` -- Adds a directory to the module search path for `import` resolution.
- `@includePath "path"` -- Alias for `@modulePath`. Adds a directory to the module/include search path.
- `@unitTestMode on` -- Enables the built-in unit testing framework. Required for `test` blocks.
- `@subsystem type` -- Sets the application subsystem. Valid values: `console` (default), `gui`.
- `@target platform` -- Sets the target platform. Valid values: `win64`, `linux64`.
- `@optimize level` -- Sets optimization level. Valid values: `debug`, `releasesafe`, `releasefast`, `releasesmall`.
- `@addVerInfo` -- Enables version information embedding in the executable.
- `@viMajor number` -- Major version number.
- `@viMinor number` -- Minor version number.
- `@viPatch number` -- Patch version number.
- `@viProductName "name"` -- Product name in version info.
- `@viDescription "text"` -- File description in version info.
- `@viFilename "name"` -- Original filename in version info.
- `@viCompanyName "name"` -- Company name in version info.
- `@viCopyright "text"` -- Copyright string in version info.

Statement-level directives:

- `@breakpoint` -- Marks a debugger breakpoint location.
- `@message hint|warn|error|fatal "text"` -- Emits a compiler diagnostic at parse time.

Predefined Symbols

Symbol	Defined when target is
`MYRA`	Always
`TARGET_WIN64`	`win64`
`WIN64`	`win64`
`MSWINDOWS`	`win64`
`WINDOWS`	`win64`
`TARGET_LINUX64`	`linux64`
`LINUX`	`linux64`
`POSIX`	`linux64`
`UNIX`	`linux64`
`CPUX64`	`win64`, `linux64`

8. Declarations

```text
Declaration     = [ "exported" ] ( ConstSection | TypeSection | VarSection | RoutineDecl ) .

ConstSection    = "const" { [ "exported" ] ConstDecl } .
ConstDecl       = ident [ ":" TypeExpr ] "=" Expression ";" .

TypeSection     = "type" { [ "exported" ] TypeDecl } .
TypeDecl        = ident "=" TypeDef ";" .

VarSection      = "var" { [ "exported" ] VarDecl } .
VarDecl         = ident ":" TypeExpr [ "=" Expression ] ";" [ ExternalVarClause ] .
ExternalVarClause = "external" [ cstring | ident ] ";" .
```

9. Routine Declarations

```text
RoutineDecl     = "routine" [ LinkageSpec ] ident [ FormalParams ] [ ":" TypeExpr ] ";"
                  ( ExternalClause | RoutineBody ) .

LinkageSpec     = '"C"' .

FormalParams    = "(" [ ParamList ] ")" .
ParamList       = ParamDecl { ";" ParamDecl } [ ";" "..." ] | "..." .
ParamDecl       = [ "var" | "const" ] ident ":" TypeExpr .

ExternalClause  = "external" [ cstring | ident ] ";" .

RoutineBody     = [ "type" { TypeDecl } ]
                  [ "const" { ConstDecl } ]
                  [ "var" { VarDecl } ]
                  "begin" StatementSeq "end" ";" .
```

- **C++ linkage (default)**: Routines use Itanium ABI name mangling, enabling overloading and type-safe linking.
- **C linkage (`"C"`)**: Disables name mangling for interoperability with C code. No overloading permitted.

10. Type Definitions

```text
TypeDef         = RecordType | ObjectType | OverlayType | ArrayType
                | PointerType | SetType | ChoicesType | RoutineType | TypeExpr .

RecordType      = "record" [ "(" TypeExpr ")" ] [ "packed" ] [ "align" "(" integer ")" ]
                  { FieldDecl | AnonOverlay } "end" .

ObjectType      = "object" [ "(" TypeExpr ")" ] { FieldDecl | MethodDecl } "end" .

OverlayType     = "overlay" { FieldDecl | AnonRecord } "end" .
AnonRecord      = "record" [ "packed" ] { FieldDecl | AnonOverlay } "end" ";" .
AnonOverlay     = "overlay" { FieldDecl | AnonRecord } "end" ";" .

FieldDecl       = ident ":" TypeExpr [ ":" integer ] ";" .

MethodDecl      = "method" ident [ FormalParams ] [ ":" TypeExpr ] ";"
                  [ "var" { VarDecl } ] "begin" StatementSeq "end" ";" .

ArrayType       = "array" [ "[" [ ArrayBounds ] "]" ] "of" TypeExpr .
ArrayBounds     = integer ".." integer .

PointerType     = "pointer" [ "to" [ "const" ] TypeExpr ] .

SetType         = "set" [ "of" ( integer ".." integer | TypeExpr ) ] .

ChoicesType     = "choices" "(" ChoicesValue { "," ChoicesValue } ")" .
ChoicesValue    = ident [ "=" Expression ] .

RoutineType     = "routine" [ LinkageSpec ] "(" [ ParamList ] ")" [ ":" TypeExpr ] .

TypeExpr        = QualIdent
                | "pointer" [ "to" [ "const" ] TypeExpr ]
                | "array" [ "[" [ ArrayBounds ] "]" ] "of" TypeExpr
                | "set" [ "of" ( integer ".." integer | TypeExpr ) ] .

QualIdent       = ident { "." ident } .
```

> **Note:** `object` replaces `class`, `choices` replaces `enum`, and `overlay` replaces `union` to avoid C++ keyword collisions. Anonymous overlays and records can nest inside each other for C data interop.
>
> **Implementation note:** The .mor grammar supports two levels of anonymous nesting (e.g., record containing anonymous overlay containing anonymous record). Deeper nesting is not currently supported. RecordLiteral (`TypeName(field: value, ...)`) is defined in the BNF but not implemented because it requires lookahead that the .mor grammar engine does not support.

11. Statements

```text
StatementSeq    = { Statement } .

Statement       = [ Assignment | CallStmt | IfStmt | WhileStmt | ForStmt
                | RepeatStmt | MatchStmt | ReturnStmt | GuardStmt | RaiseStmt
                | CreateStmt | DestroyStmt
                | GetMemStmt | FreeMemStmt | ResizeMemStmt | SetLengthStmt
                | WriteStmt | TestAssertStmt | Directive | ";" ] .

Assignment      = Designator ( ":=" | "+=" | "-=" | "*=" | "/=" ) Expression [ ";" ] .

CallStmt        = Designator [ ";" ] .

IfStmt          = "if" Expression "then" StatementSeq [ "else" StatementSeq ] "end" [ ";" ] .

WhileStmt       = "while" Expression "do" StatementSeq "end" [ ";" ] .

ForStmt         = "for" ident ":=" Expression ( "to" | "downto" ) Expression
                  "do" StatementSeq "end" [ ";" ] .

RepeatStmt      = "repeat" StatementSeq "until" Expression [ ";" ] .

MatchStmt       = "match" Expression "of" { MatchArm } [ "else" StatementSeq ] "end" [ ";" ] .
MatchArm        = MatchLabel { "," MatchLabel } ":" StatementSeq .
MatchLabel      = Expression [ ".." Expression ] .

ReturnStmt      = "return" [ Expression ] [ ";" ] .

GuardStmt       = "guard" StatementSeq
                  ( "except" StatementSeq [ "finally" StatementSeq ]
                  | "finally" StatementSeq ) "end" [ ";" ] .

RaiseStmt       = ( "raiseException" "(" Expression ")"
                  | "raiseExceptionCode" "(" Expression "," Expression ")" ) [ ";" ] .

CreateStmt      = "create" "(" Expression ")" [ ";" ] .
DestroyStmt     = "destroy" "(" Expression ")" [ ";" ] .
GetMemStmt      = "getMem" "(" Expression ")" [ ";" ] .
FreeMemStmt     = "freeMem" "(" Expression ")" [ ";" ] .
ResizeMemStmt   = "resizeMem" "(" Expression "," Expression ")" [ ";" ] .
SetLengthStmt   = "setLength" "(" Expression "," Expression ")" [ ";" ] .
WriteStmt       = ( "write" | "writeln" ) "(" [ ArgList ] ")" [ ";" ] .

TestAssertStmt  = ( "testAssert" | "testAssertTrue" | "testAssertFalse"
                | "testAssertNil" | "testAssertNotNil" ) "(" Expression ")" [ ";" ]
                | ( "testAssertEqualInt" | "testAssertEqualUInt"
                | "testAssertEqualFloat" | "testAssertEqualStr"
                | "testAssertEqualBool" | "testAssertEqualPtr" )
                  "(" Expression "," Expression ")" [ ";" ]
                | "testFail" "(" Expression ")" [ ";" ] .
```

> **Note:** C++ statements (`using namespace std;`, `#include "file.h"`, etc.) are handled by Metamorf's built-in C++ passthrough layer and do not need to be defined in Myra's grammar. Any C++ keyword appearing in statement position is collected verbatim and emitted to the C++ output. See the Metamorf documentation for details.

12. Expressions

```text
Expression      = OrExpr .
OrExpr          = AndExpr { "or" AndExpr } .
AndExpr         = CompExpr { ( "and" | "xor" ) CompExpr } .
CompExpr        = SimpleExpr [ RelOp SimpleExpr ] .
RelOp           = "=" | "<>" | "<" | ">" | "<=" | ">=" | "in" .

SimpleExpr      = [ "+" | "-" ] ShiftExpr { AddOp ShiftExpr } .
AddOp           = "+" | "-" .

ShiftExpr       = Term { ShiftOp Term } .
ShiftOp         = "shl" | "shr" .

Term            = Factor { MulOp Factor } .
MulOp           = "*" | "/" | "div" | "mod" .

Factor          = "not" Factor | "-" Factor | "+" Factor
                | "address" "of" Factor | Primary .

Primary         = integer | float_literal | cstring | wstring
                | "true" | "false" | "nil"
                | SetLiteral | RecordLiteral
                | "(" Expression ")" | Designator | Intrinsic | TypeCast .

Designator      = ( ident | "self" | "parent" | "varargs" ) { Selector } .
Selector        = "." ident | "[" Expression "]" | "^" | "(" [ ArgList ] ")" .

ArgList         = Expression { "," Expression } .

SetLiteral      = "[" [ SetElement { "," SetElement } ] "]" .
SetElement      = Expression [ ".." Expression ] .

RecordLiteral   = ident "(" FieldInit { "," FieldInit } ")" .
FieldInit       = ident ":" Expression .

TypeCast        = TypeExpr "(" Expression ")" .
```

Pointer Operations

- `address of expr` -- Returns a pointer to the operand.
- `expr^` -- Postfix (selector): dereference. Follows the pointer to its target.

> **Note:** C++ scope resolution (`::`) and arrow access (`->`) are handled by Metamorf's C++ passthrough layer and work naturally in expressions alongside Myra's own operators.

13. Intrinsics

```text
Intrinsic       = LenExpr | SizeExpr | Utf8Expr | ParamCountExpr | ParamStrExpr
                | GetExceptionCodeExpr | GetExceptionMessageExpr .

LenExpr                  = "len" "(" Expression ")" .
SizeExpr                 = "size" "(" ( TypeExpr | Expression ) ")" .
Utf8Expr                 = "utf8" "(" Expression ")" .
ParamCountExpr           = "paramCount" "(" ")" .
ParamStrExpr             = "paramStr" "(" Expression ")" .
GetExceptionCodeExpr     = "getExceptionCode" "(" ")" .
GetExceptionMessageExpr  = "getExceptionMessage" "(" ")" .
```

> **Note:** `size` replaces `sizeof` to avoid collision with C++ `sizeof`.
> Memory management (`create` / `destroy` / `getMem` / `freeMem` / `resizeMem` / `setLength`)
> is defined in Statements (Section 11), not as expression-level intrinsics.

14. Variadic Arguments

```text
ParamList       = ParamDecl { ";" ParamDecl } [ ";" "..." ] | "..." .

VarArgsAccess   = "varargs" "." "next" "(" TypeExpr ")"
                | "varargs" "." "copy" "(" ")"
                | "varargs" "." "count" .
```

- `varargs.next(TypeExpr)` -- Retrieves and consumes the next variadic argument.
- `varargs.count` -- Total number of variadic arguments passed.
- `varargs.copy()` -- Returns a new `varargs` object with a copied cursor position.

15. Operator Precedence (Highest to Lowest)

Precedence	Operators
1 (highest)	`not` `-` (unary) `+` (unary) `address of`
2	`*` `/` `div` `mod`
3	`shl` `shr`
4	`+` `-`
5	`=` `<>` `<` `>` `<=` `>=` `in`
6	`and` `xor`
7 (lowest)	`or`

16. C++ Interop

Myra is built with Metamorf, which provides seamless C++ passthrough as an inherited capability. This means every Myra source file can freely mix Myra code and raw C++ code without any special syntax or escape mechanism.

What Myra defines (semantically analyzed): all keywords, types, and grammar constructs listed in this BNF, including variable and routine declarations, control flow, exception handling (`guard` / `except` / `finally`), memory management (`create` / `destroy` / `getMem` / `freeMem` / `resizeMem`), type checking, scope analysis, and duplicate detection.

What C++ provides (passthrough, no semantic analysis by Myra): all C++ keywords and constructs (`class` , `struct` , `enum` , `template` , `namespace` , etc.), `{ }` blocks, `::` qualified names, `->` arrow access, preprocessor directives (`#include` , `#define`), and standard C++ exception syntax (`try` / `catch` / `throw`).

The C++ passthrough layer is not part of Myra's grammar. It is a Metamorf platform feature inherited by all languages built on Metamorf. See the Metamorf documentation for full details on how C++ passthrough tokenization, parsing, and code generation work.

Myra™ Programming Language.

Copyright © 2025-present tinyBigGAMES™ LLC
All Rights Reserved.